Bitácora de construcción

Registro continuo de lo que se ha agregado al proyecto: qué es cada pieza, cómo funciona por dentro, qué significan los nombres y variables que aparecen en el código, por qué se eligió así y qué idea de C#, PostgreSQL o de herramientas enseña. Léela una vez de principio a fin y después actualízala con cada pull request.

English version: BUILD-LOG.md. Los PDF/Word exportados están en `exports/`.

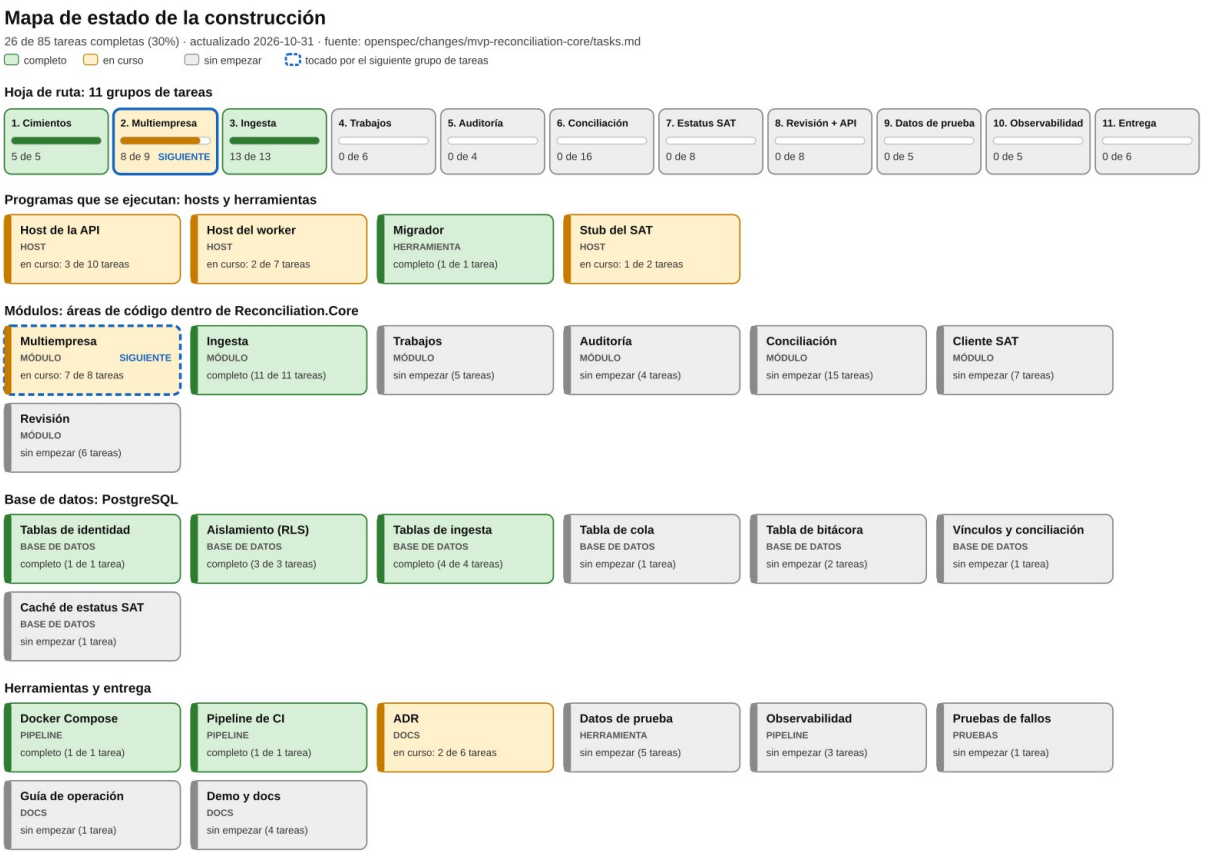
Fuente de verdad de los elementos de trabajo: `openspec/changes/mvp-reconciliation-core/tasks.md` (85 tareas en 11 grupos). Esta bitácora las explica; el mapa de estado de abajo se genera a partir de ellas.

Cómo está organizada la documentación

Carpeta	Qué contiene	Por qué está separada
`docs/adr/`	Registros de decisiones de arquitectura (ADR-001 a ADR-003)	Una decisión = un archivo; nunca se editan, se reemplazan por otro ADR
`docs/build-log/`	Esta bitácora en español e inglés, y `exports/` con los PDF/Word generados	Lo que se actualiza en cada PR; los exportados no se mezclan con el texto fuente
`docs/requirements/`	BRS y SRS (inglés y español)	Documentos de requisitos que casi no cambian
`docs/reference/`	Formato CSV bancario de referencia y la ficha del proyecto con enlaces de consulta	Material de consulta que las pruebas leen o que se cita
`docs/diagrams/`	Diagramas de base de datos, de roles y el mapa de progreso (`progress/`)	Imágenes generadas o dibujadas, con su `.svg` fuente
`openspec/`	La especificación y las tareas del cambio `mvp-reconciliation-core`	Fuente de verdad del alcance

Regla práctica: antes de borrar o mover un archivo de `docs/`, busca su nombre con `grep -rn` (varias pruebas xUnit leen `docs/adr/`, `docs/reference/` y `docs/build-log/`), y usa `git mv` para conservar el historial.

1. Estado de un vistazo



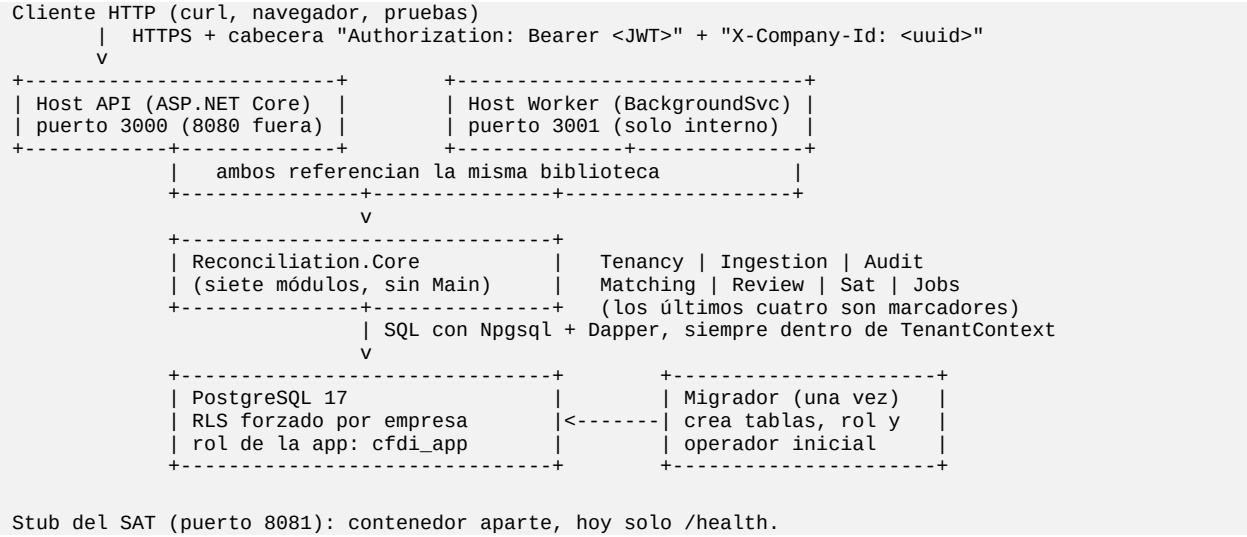
26 de 85 tareas completas. Grupo 3 completo; del grupo 2 solo queda 2.4 para decisiones reales del grupo 8. Regenera la imagen después de marcar tareas: `python3 tools/progress_diagram.py --lang es` (sin `--lang es` genera la versión en inglés).

Qué significa cada tipo

Tipo	Qué es	Ejemplo aquí
Host	Un programa que sigue ejecutándose y atiende solicitudes o hace trabajo en segundo plano	API, worker, stub del SAT
Módulo	Un área de código dentro de la biblioteca compartida, con una responsabilidad de negocio	Conciliación, Auditoría
Base de datos	Tablas, reglas y seguridad que viven dentro de PostgreSQL	`company`, seguridad a nivel de fila
Herramienta	Un programa que se ejecuta una vez para hacer un trabajo y termina	Migrador, generadores de datos de prueba
Pipeline	Automatización que compila, prueba o arranca cosas	Docker Compose, CI
Docs / Pruebas	Decisiones escritas, o código que verifica al resto del código	ADR, pruebas xUnit

2. Arquitectura

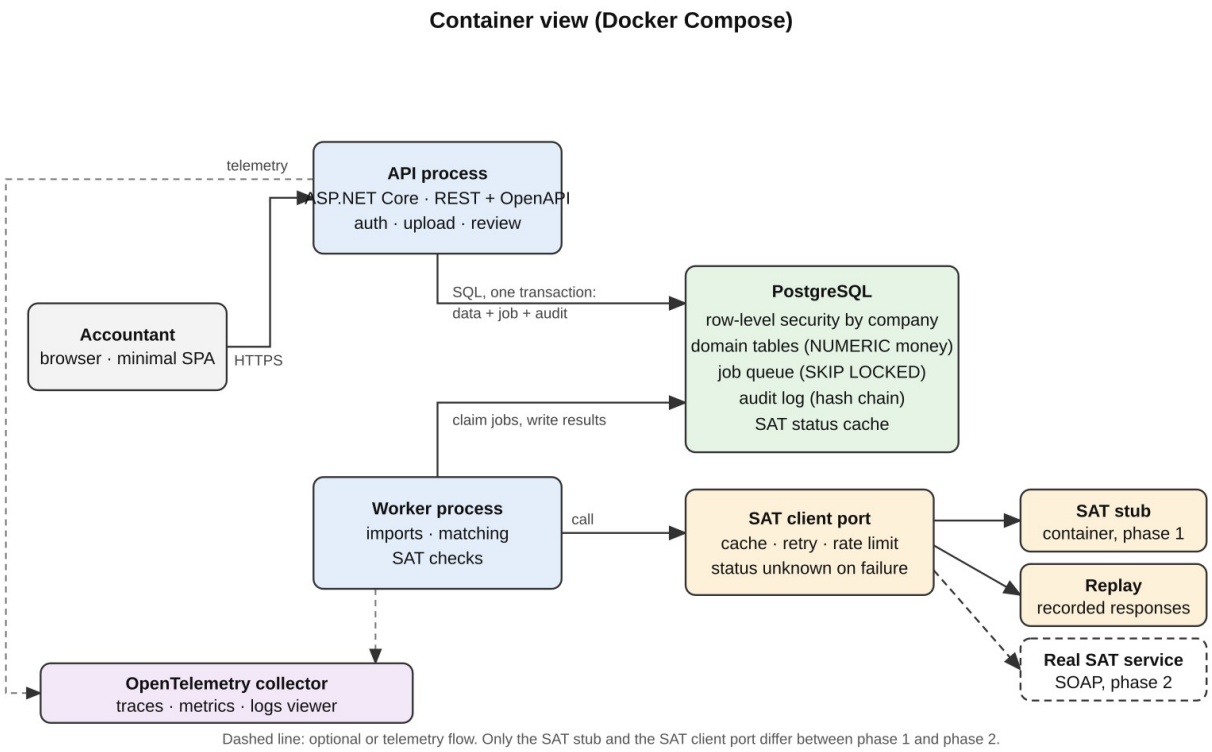
2.1 Vista general: qué está construido hoy



Cada flecha es una frontera con una razón de ser:

- **Dos hosts, una biblioteca.** La API atiende personas y debe responder rápido; el worker hará tareas largas (conciliar, consultar al SAT). Comparten `Reconciliation.Core`, así la regla de negocio se escribe una sola vez (ADR-001: monolito modular con worker separado, en lugar de microservicios).
- **Módulos con fronteras.** `Tenancy` (empresas, usuarios, roles), `Ingestion` (lectura de CFDI y CSV), `Audit` (bitácora), y los marcadores `Matching`, `Review`, `Sat`, `Jobs` que se llenarán en los grupos 4 a 8. Un módulo habla con otro por interfaces, nunca leyendo las tablas ajenas.
- **PostgreSQL como árbitro final.** Aunque el código olvide filtrar por empresa, la base de datos no entrega filas de otra empresa (sección 2.3).
- **El migrador es un programa aparte** con otro rol de base de datos: el dueño del esquema (`cfdi`) crea y cambia tablas; la aplicación (`cfdi_app`) solo puede lo que se le concede.

El diseño completo, incluido lo que aún no existe (cola de trabajos, cliente SAT, telemetría), es este:



Lo construido hasta ahora es la caja *API process*, el *Worker* (solo arranca y reporta salud), *PostgreSQL* con el aislamiento por empresa y las tablas de ingesta, y el *SAT stub* como marcador. Faltan la cola de trabajos (`SKIP LOCKED`), la cadena de hash de auditoría, el puerto del cliente SAT y OpenTelemetry.

2.2 Ciclo de una solicitud autenticada

Ejemplo: un contador sube una factura con `POST /companies/{id}/imports/cfdi`.

1. **Login.** El cliente llama `POST /auth/tokens` con `{"email","password"}`. La API busca el usuario sin distinguir mayúsculas, verifica la contraseña (PBKDF2 con sal) y devuelve un JWT firmado (HMAC-SHA256) válido una hora. El token solo lleva el identificador del usuario en el *claim* `sub`.
2. **Solicitud.** El cliente envía el archivo con `Authorization: Bearer <token>` y `X-Company-Id: <uuid>`.
3. **Autenticación.** ASP.NET valida firma, emisor, audiencia y caducidad. Si falla, responde 401.
4. **Autorización por empresa.** `CompanyAuthorization` exige que el `X-Company-Id` coincida con el `{id}` de la ruta, abre una transacción con ese contexto y lee de `membership` el rol del usuario. La política `accountant` acepta administrador y contador; consulta (`viewer`) recibe 403.
5. **Contexto de empresa.** `TenantContext.ExecuteAsync` abre una conexión, inicia una transacción y ejecuta `set_config('app.company_id', <uuid>, true)`; el `true` hace que valga solo dentro de esa transacción.
6. **Trabajo.** El importador lee y valida el archivo, e inserta filas; cada consulta pasa por las políticas RLS.
7. **Cierre.** Si todo salió bien, `COMMIT`; si algo falla, `ROLLBACK` y no queda nada a medias. La respuesta es un `ImportReport` con un resultado por archivo.

2.3 Capas de seguridad (defensa en profundidad)

Capa	Qué impide	Dónde vive	Cómo se prueba
Autenticación JWT	Solicitudes anónimas	`Authentication.cs`, política de respaldo	`AuthenticationTests`
Membresía y rol por empresa	Usar una empresa donde no se tiene rol, o una acción no permitida al rol	`CompanyAuthorization.cs`	`MembershipTests`, `IsolationTests`
Transacción con contexto	Consultas sin empresa asignada	`TenantContext.cs`	`TenantContextTests`
RLS forzado en PostgreSQL	Ver o escribir filas de otra empresa aunque falte un `WHERE`	Migraciones `M0002` y `M0004`	Guardián de aislamiento sobre todas las tablas y rutas
Claves foráneas compuestas	Que una fila apunte a un registro de otra empresa	`FOREIGN KEY (company_id, x_id)`	`IngestionDatabaseTests`
Rol sin privilegios	Saltarse RLS, borrar datos	`cfdi_app`: sin superusuario, sin `BYPASSRLS`, sin `DELETE`	Pruebas que usan el rol real
Lectura segura de XML	Ataques XXE y archivos "bomba"	`SafeCfdiXml.cs`	`SafeXmlTests`

Por qué varias capas: cada una cubre un fallo distinto. Un 403 no demuestra que exista la política RLS, y una política RLS no identifica al usuario; por eso hacen falta ambas y pruebas para cada una.

2.4 Ciclo de una importación

```
multipart (files, bank, accountReference)
  -> MultipartUpload: cuenta bytes mientras lee (413/400 si pasa los límites)
  -> por cada archivo:   SafeCfdiXml -> CfdiParser -> ¿involucra a la empresa? -> ¿UUID repetido?
  -> por cada archivo, UNA transacción propia:
       import_file (original, sha256, retained_until) -> cfdi/pagos/movimientos -> audit
  -> ImportReport { upload_id, archivos[ estado, motivo, ids ] }
```

El punto clave es "una transacción por archivo": si el tercer archivo de diez es inválido, los otros nueve se importan y el tercero queda como `rejected` con su motivo. Todos los archivos de una subida comparten `upload_id` (un UUID nuevo por subida) para poder pedir luego el reporte completo.

2.5 Modelo de datos

Tabla	Para qué existe	Columnas clave
`company`	Cada cliente (empresa) del sistema	`rfc` (inmutable, validado), `date_tolerance_days`, `amount_tolerance`, `archived_at`
`app_user`	Personas que inician sesión	`email` (único sin distinguir mayúsculas), `password_hash`
`membership`	Une usuario, empresa y rol	PK `(company_id, user_id)`, `role` (`administrator`, `accountant`, `viewer`)
`audit_entry`	Qué hizo quién y cuándo	`actor_id`, `action`, `old_values`, `new_values` (jsonb)
`import_file`	Un archivo subido y su resultado	`upload_id`, `status`, `reason`, `original` (bytes), `sha256`, `retained_until`, `original_file_id`
`cfdi`	Una factura importada	`uuid`, `type` (I/P/E), `direction`, importes `numeric(18,2)`, `is_global`, `review_reason`

Tabla	Para qué existe	Columnas clave
`cfdi_relation`	Relaciones entre CFDI (por ejemplo, nota de crédito)	`related_uuid`, `relation_code`
`cfdi_payment`, `cfdi_payment_doc`	Complemento de pago y las facturas que liquida	`payment_index`, `installment`, `previous_balance`, `paid`, `remaining_balance`
`bank_account`	Cuenta bancaria identificada por etiqueta	`bank`, `account_reference`
`bank_movement`	Un renglón del estado de cuenta	`movement_date`, `credit`, `debit`, `balance`, `reference`

Convenciones que se repiten en todas las tablas de empresa: `id uuid` generado por la base; `company_id` obligatorio; RLS habilitado **y** forzado; claves foráneas compuestas; dinero en `numeric(18,2)`, nunca `double` ni `float`.

3. Diccionario de nombres y variables

3.1 Variables de entorno y configuración

Nombre	Valor de ejemplo	Quién lo lee	Qué significa
`ConnectionStrings__Default`	`Host=postgres;Port=5432;Username=cfdi_app;...`	API, worker, migrador	Cadena de conexión a PostgreSQL. El doble guion bajo es cómo .NET escribe `ConnectionStrings:Default` en una variable de entorno. API y worker usan el rol `cfdi_app`; el migrador usa el dueño `cfdi`
`APP_DB_PASSWORD`	marcador sintético	Migrador y Compose	Contraseña con la que el migrador activa el login del rol `cfdi_app` (`ALTER ROLE ... LOGIN PASSWORD`)
`Jwt__SigningKey` / `JWT_SIGNING_KEY`	al menos 32 bytes	API	Clave secreta con la que se firman y verifican los tokens. Si falta o es corta, la API no arranca. `JWT_SIGNING_KEY` es el nombre en `.env` para Compose, que lo entrega como `Jwt__SigningKey`
`OPERATOR_EMAIL`, `OPERATOR_PASSWORD`	opcionales; contraseña de 12+ caracteres	Migrador (`up`)	Si existen, se crea una sola vez el operador de plataforma; nunca se restablece la contraseña de uno existente
`Ingestion__MaxFileBytes`	10485760 (10 MiB)	Ingesta	Tamaño máximo de un archivo. Uno mayor se rechaza solo a él
`Ingestion__MaxUploadBytes`	20971520 (20 MiB)	Ingesta	Tamaño máximo de toda la subida, contando cabeceras multipart. Excederlo rechaza toda la carga (413)
`Ingestion__MaxFiles`	20	Ingesta	Cantidad máxima de archivos por subida (400 si se excede)
`Ingestion__MaxXmlNodes`	100000	Ingesta	Nodos XML máximos; frena archivos "bomba"
`Ingestion__MaxXmlDepth`	64	Ingesta	Profundidad máxima de anidación XML
`Ingestion__RetentionYears`	10 (mínimo 5, máximo 100)	Ingesta	Años que se conserva el original; fija `retained_until`

`IngestionOptions.Validate()` revisa todo al arrancar: un valor no positivo o una retención fuera de 5–100 impide iniciar, en lugar de fallar después en medio de una importación.

3.2 Cabeceras, rutas y campos HTTP

Nombre	Dónde aparece	Significado
`Authorization: Bearer <JWT>`	Cabecera	El token obtenido en el login
`X-Company-Id`	Cabecera	La empresa en la que actúa la solicitud; debe ser la misma del `{id}` de la ruta y el usuario debe tener membresía
`{id}` / `companyId`	Ruta / código	UUID de la empresa
`{uploadId}` / `upload_id`	Ruta / columna	UUID que agrupa los archivos de una misma subida
`{invoiceId}`	Ruta	UUID interno (`cfdi.id`) de la factura. No es el UUID fiscal
`{fileId}`	Ruta	UUID de una fila de `import_file`
`files`	Campo multipart	Uno o más archivos subidos
`bank`	Campo multipart	Identificador del formato bancario; hoy solo `reference`
`accountReference`	Campo multipart	Etiqueta de 1–40 letras/dígitos ASCII, espacios, guiones y guiones bajos. Nunca un número de cuenta real

3.3 Nombres de base de datos

Nombre	Significado
`app.company_id`	Variable de sesión de PostgreSQL que contiene la empresa actual. Las políticas RLS la comparan con `company_id`
`cfdi` (usuario)	Dueño del esquema; solo lo usa el migrador. Puede saltarse RLS, por eso la aplicación no lo usa
`cfdi_app` (rol)	Rol de la aplicación: `NOSUPERUSER NOBYPASSRLS`, solo `SELECT/INSERT` y `UPDATE` en columnas concretas
`membership_role`	Tipo enumerado: `administrator`, `accountant`, `viewer`
`uuid` (en `cfdi`)	Identificador fiscal del CFDI (folio fiscal del SAT). Es único **por empresa**
`type`	`I` ingreso, `E` egreso (nota de crédito), `P` pago (complemento de pago)
`direction`	`issued` si la empresa es el emisor; `received` si es el receptor
`status` (en `import_file`)	`imported`, `already_imported` (UUID repetido) o `rejected`
`original_file_id`	En un duplicado, apunta al archivo que ya se había aceptado
`sha256`	Huella de 32 bytes del archivo; permite probar que no cambió
`retained_until`	Fecha antes de la cual el original no puede borrarse ni acortarse
`date_tolerance_days`	Días de diferencia admitidos entre fecha bancaria y fiscal al conciliar (por defecto 3)
`amount_tolerance`	Diferencia de importe admitida al conciliar (por defecto 0.01)
`complement_next_biz_day`	Regla de plazo para complementos de pago; se usará en el grupo 6
`numeric(18,2)`	Dinero con 18 dígitos y 2 decimales exactos
`numeric(20,10)`	Tipo de cambio con hasta 10 decimales

3.4 Nombres frecuentes en el código C#

Nombre	Significado
`actorId`	UUID del usuario que ejecuta la acción; sale del claim `sub` y se guarda en la auditoría
`uploadId`	UUID generado por cada subida (`Guid.NewGuid()`)
`UploadedFile(Name, Content, Hash, Error)`	Un archivo en memoria con su huella SHA-256; `Error` lleva el motivo si no se pudo leer
`FileImportResult`	Resultado de un archivo: estado, motivo, ids y conteos de movimientos
`ImportReport(UploadId, Files)`	Lo que responde la API: todos los resultados de una subida
`ParsedCfdi`, `PaymentDocument`	Representación ya leída de un CFDI y de cada línea de pago
`InconsistentBalance`	Marca de una línea de pago cuyo saldo no encaja; no rechaza, manda a revisión
`IBankStatementParser`	Contrato para leer el CSV de un banco; hay un implementador por banco
`TenantContext`	Servicio que ejecuta código dentro de una transacción con la empresa asignada

4. Qué existe hoy

Pieza	Tipo	Para qué sirve	Dónde
Archivo de solución	Compilación	Agrupa los seis proyectos para que un solo `dotnet build` compile todo	`Reconciliation.slnx`
Reglas de compilación compartidas	Compilación	Todos los proyectos usan la misma configuración: .NET 10, verificación de nulos, las advertencias son errores	`Directory.Build.props`, `.editorconfig`, `global.json`
Biblioteca Core	Área de módulos	El código que comparten ambos hosts: multiempresa, ingesta, auditoría mínima y código común de base de datos y salud	`src/Reconciliation.Core`
Host de la API	Host	La API web que llamarán los usuarios. Atiende autenticación, empresas, miembros, configuración, archivo, importaciones, salud y OpenAPI	`src/Reconciliation.Api`
Host del worker	Host	Ejecuta trabajos en segundo plano. Hoy arranca, reporta su salud y espera	`src/Reconciliation.Worker`
Migrador	Herramienta	Aplica o revierte cambios de la base de datos (`up`, `down`) y crea el operador inicial	`src/Reconciliation.Migrator`
Stub del SAT	Host	Simulación del servicio de estatus del SAT para desarrollar sin cuenta real. Hoy es solo un marcador	`src/Reconciliation.SatStub`
PostgreSQL 17	Base de datos	Guarda todos los datos con RLS forzado por empresa	`docker-compose.yml`, migraciones `M0001` a `M0004`
Docker Compose	Pipeline	Arranca base de datos, migrador, API, worker y stub con un solo comando y espera a que todos estén sanos	`docker-compose.yml`, `Dockerfile`
Pipeline de CI	Pipeline	En cada push: formato, compilación, pruebas, migraciones subir/bajar/subir y arranque completo del sistema	`.github/workflows/ci.yml`
ADR	Docs	Decisiones grandes y alternativas descartadas	`docs/adr/`

Pieza	Tipo	Para qué sirve	Dónde
Requisitos y plan	Docs	BRS, SRS, diagramas y el cambio de OpenSpec que define el trabajo	`docs/requirements/`, `docs/diagrams/`, `openspec/`
Pruebas	Pruebas	104 casos xUnit: 101 aprobados, 3 pendientes explícitos; PostgreSQL real para multiempresa e ingesta	`tests/ Reconciliation.Tests`
Mapa de progreso	Herramienta	Dibuja la imagen de estado a partir de `tasks.md`	`tools/ progress_diagram.py`

5. Registro de pull requests

Primero el más reciente. Cada entrada responde seis preguntas: **Qué es** (la pieza), **Cómo funciona** (paso a paso),

Nombres y variables (qué significa cada término nuevo), **Por qué así** (decisión y alternativa descartada),

Qué aprendes (idea de C#, SQL o herramientas) y **Verificado** (cómo se comprobó).

PR-3: Modelo de dominio e ingesta (3.1–3.8, incluidas 3.4b–3.4f)

Qué es: la entrada de datos al sistema. Convierte CFDI sintéticos (XML) y estados de cuenta CSV de referencia en registros aislados por empresa, con importes exactos y con el archivo original conservado.

Cómo se construyó: cada implementación sigue a su commit de pruebas rojas. La persistencia usa PostgreSQL real mediante Testcontainers y el migrador real; los parsers también se prueban directamente con entradas inválidas y hostiles.

Verificado: compilación sin advertencias/errores, `dotnet test --solution Reconciliation.slnx` con 101 aprobados y 3 omitidos, revisión de formato, Compose compilado y sano, migraciones subir/bajar/subir en una base desechable y ambos mapas regenerados. No se hizo push.

Modelo de datos, dinero exacto y RLS (3.2, 3.8)

- **Qué es:** la migración `M0004Ingestion` y las siete tablas nuevas: `import_file`, `cfdi`, `cfdi_relation`, `cfdi_payment`, `cfdi_payment_doc`, `bank_account` y `bank_movement` (ver sección 2.5).
- **Cómo funciona:** todas llevan `company_id`, RLS habilitado y forzado y una política `tenant_isolation`. Las claves foráneas son compuestas, por ejemplo `(company_id, cfdi_id)`, así una fila de la empresa A no puede apuntar a una factura de la B ni por error. El UUID fiscal es único por empresa (`UNIQUE(company_id, uuid)`). La identidad de un movimiento bancario es la combinación cuenta + fecha + descripción + abono + cargo + saldo, y un `INSERT ... ON CONFLICT DO NOTHING` convierte una repetición en "duplicado" en vez de error.
- **Nombres y variables:** `numeric(18,2)` para dinero, `numeric(20,10)` para tipos de cambio; `CHECK(total = subtotal
  - discount + transferred_tax - retained_tax + local_transferred_tax - local_retained_tax)` obliga a que el total de la cabecera cuadre con sus partes.
- **Por qué así:** una restricción `CHECK` protege incluso contra un error del código C#; PostgreSQL es la última barrera. Se descartó `double`: no representa 0.10 exactamente. Con `decimal` en C# y `numeric` en PostgreSQL, 0.10 + 0.20 da exactamente 0.30.

- **Qué aprendes:** claves foráneas compuestas como técnica de aislamiento; `GRANT UPDATE(columna)` para dar permiso solo sobre columnas concretas; `ON CONFLICT` para operaciones idempotentes.

XML seguro y valores fiscales declarados (3.3, 3.4, 3.4b–3.4e)

- **Qué es:** `SafeCfdiXml` (lectura segura) y `CfdiParser` (interpretación del CFDI 4.0 y del complemento de pago 2.0).
- **Cómo funciona:** primero un recorrido acotado cuenta nodos y profundidad (`MaxXmlNodes`, `MaxXmlDepth`) con `DtdProcessing.Prohibit` y `XmlResolver = null`, lo que impide declarar entidades externas (XXE) y expandir entidades (billion laughs). Solo si pasa, se carga el documento. Después el parser valida:
  - La aritmética de cabecera incluye descuentos, traslados y retenciones federales y locales, **sin recalcular tasas**: se verifican los valores declarados, no se rehacen las reglas fiscales.
  - La muestra sintética importa con total 1,939.27. Importes de conceptos con seis decimales se aceptan sin recalcularlos ni guardarlos por separado. Cabecera, pagos y saldos con más de dos decimales declarados se rechazan, incluidos ceros adicionales; el MVP trata otras monedas con dos decimales.
  - Versiones antiguas del complemento de pago se rechazan.
  - Cada línea de pago comprueba `saldo anterior - pagado = saldo restante` y la continuidad entre parcialidades consecutivas presentes en la carga. Una inconsistencia se guarda con `inconsistent_balance = true` y un `review_reason`, **sin rechazar** el complemento. No se infieren parcialidades ausentes ni se exige haber importado la factura pagada.
  - Se guardan códigos de relación incluso desconocidos, el indicador de factura global/RFC genérico y las fechas de emisión y timbrado. Ingesta conserva las relaciones; no las aplica a importes pendientes (eso es del grupo 6).
- **Nombres y variables:** `InvoiceUuid`, `Installment` (número de parcialidad), `PreviousBalance`, `Paid`, `RemainingBalance` son los campos de cada línea de pago; `ReviewReason` explica por qué una factura requiere revisión.
- **Por qué así:** archivos rechazados o ajenos a la empresa no afectan a las líneas válidas. Rechazar más de dos decimales declarados evita redondear en silencio un importe fiscal.
- **Qué aprendes:** verificar valores declarados es distinto de recalcular reglas fiscales. Las fechas locales no llevan zona horaria y admiten hasta seis dígitos fraccionarios; se rechaza más precisión para no cambiarla al guardarla en PostgreSQL. `LINQ to XML` (`XDocument`) y `XmlReaderSettings` son la forma segura de leer XML en .NET.

Reportes, originales y límites (3.4f, 3.5)

- **Qué es:** `CfdiImporter`, `ImportStorage` y `MultipartUpload`: guardan cada archivo con su resultado y leen la subida con límites.
- **Cómo funciona:** cada archivo tiene una fila `import_file`, agrupada por `upload_id`, sin tabla de lote. Se guarda estado, motivo y SHA-256; un rechazado conserva el hash, no los bytes. Un UUID repetido apunta al original (`original_file_id`) y no lo reemplaza. Un bloqueo transaccional (`pg_advisory_xact_lock`) por empresa/UUID serializa duplicados concurrentes, respaldado por la restricción única. Un trigger impide cambiar originales/hashes o acortar la conservación; no hay privilegio `DELETE` ni ruta DELETE. `RetentionYears` vale 10 por defecto, se valida al arrancar (mínimo 5, máximo conservador 100) y fija `retained_until`.
- **Límites y decisiones:** `MaxFileBytes` = 10 MiB, `MaxUploadBytes` = 20 MiB, `MaxFiles` = 20, `MaxXmlNodes` = 100000 y `MaxXmlDepth` = 64, configurables en `Ingestion`. Tamaño agregado o cantidad excesiva rechazan toda la carga antes de persistir (413/400). Un archivo demasiado grande o

un XML inválido/hostil solo rechaza ese archivo. Se cuentan bytes durante la lectura multipart, incluso sin `Content-Length`. Los bytes de un archivo grande se descartan mientras se calcula su hash; el buffer aceptado queda dentro del máximo agregado. Lecturas y SQL reciben cancelación.

- **Por qué así:** se prefiere un hash más un rechazo claro antes que guardar basura. Un bloqueo asesor es más simple y barato que reintentar tras un error de duplicado.
- **Qué aprendes:** cada archivo necesita su transacción para que uno inválido no revierta los demás; se puede conservar el hash completo sin guardar el contenido rechazado; `CancellationToken` permite abortar trabajo si el cliente se desconecta. El encadenamiento completo de auditoría sigue en el grupo 5.

Estados de cuenta de referencia (3.1, 3.6, 3.7)

- **Qué es:** `ReferenceBankStatementParser` y `BankStatementImporter`, más el documento `docs/reference/bank-csv-reference-format.md`.
- **Cómo funciona:** la cabecera exacta es `fecha,concepto,abono,cargo,saldo`; UTF-8, fechas `yyyy-MM-dd`, importes con punto y dos decimales. Una línea inválida rechaza todo el archivo con su número de línea y no crea cuenta ni movimientos parciales. Un banco desconocido informa `reference`. `bank=reference` y `accountReference` crean o reutilizan la cuenta por empresa/banco/etiqueta; no hay CRUD de cuentas.
- **Nombres y variables:** `accountReference` admite 1–40 letras/dígitos ASCII, espacios, guiones y guiones bajos; es una etiqueta, nunca un número real completo (sugerencia: los últimos cuatro dígitos). `RASTREO:<clave>` o `REF:<número>` dentro de `concepto` llenan `reference`; RASTREO tiene prioridad; sin token la referencia es nula. `ImportedMovements` y `DuplicateMovements` cuentan filas nuevas y repetidas.
- **Por qué así:** cargas traslapadas del mismo periodo no duplican movimientos; filas iguales con saldo distinto se conservan porque son movimientos distintos. Como decisión conservadora, se rechaza un archivo sin movimientos y cada línea debe tener exactamente un abono o un cargo positivo. También se guarda el original del estado de cuenta.
- **Qué aprendes:** reutilizar el lector CSV de la plataforma evita errores con comillas y comas; una interfaz (`IBankStatementParser`) deja agregar bancos reales sin tocar al importador.

API y cierre de escenarios de multiempresa

- **Qué es:** `IngestionEndpoints`: dos `POST` de importación y cuatro `GET` de consulta.
- **Cómo funciona:** todas las rutas validan membresía y `X-Company-Id`. `POST /companies/{id}/imports/cfdi` y `/imports/bank-statement` exigen contador o administrador. `GET /companies/{id}/imports/{uploadId}`, `/cfdi/{invoiceId}`, `/cfdi/{invoiceId}/xml`, `/imports/files/{fileId}/original` y `/banks` aceptan a cualquier miembro, incluido consulta. Los archivos usan el campo multipart `files`.
- **Pruebas:** el guardián de aislamiento descubre rutas, prepara casos exitosos, intenta ids de la empresa B bajo contexto válido de A y compara todas las tablas de B antes y después. "Cross-company attempt in the test suite" ahora comprueba 404 con una factura real de B; "Delete attempt" comprueba 405 y factura/original intactos. Se completan las tareas 2.6 y 2.9.
- **Qué aprendes:** un 404 solo demuestra aislamiento después de comprobar que el propietario sí puede leer el recurso.

Aún omitidos: "Viewer cannot decide" y "Different roles" esperan vínculos reales del grupo 8; "Tolerance changed" espera una corrida del grupo 6. La tarea 2.4 sigue sin marcar. Las importaciones son síncronas hasta los trabajos del grupo 4.

PR-2: Multiempresa y acceso (grupo 2; seis tareas completas y tres pendientes)

Qué es: el aislamiento de cada empresa desde PostgreSQL, la autenticación del usuario y la gestión de acceso y configuración por administradores. Identificadores y credenciales de prueba son sintéticos.

Cómo se construyó: cada corte tuvo primero su commit de pruebas rojas y después el de implementación. Testcontainers ejecuta el migrador real contra PostgreSQL 17; la aplicación entra como `cfdi_app`, sin superusuario ni `BYPASSRLS`.

Verificado: `dotnet build` sin advertencias ni errores, `dotnet test --solution Reconciliation.slnx` con 40 casos aprobados y 5 omitidos explícitamente, `dotnet format --verify-no-changes`, Compose sano tras compilar y migraciones subir/bajar/subir en una base desechable separada. También funcionó revertir hasta cero y volver a aplicar todo.

Transacciones y RLS forzado (2.1, 2.2)

- **Qué es:** `TenantContext` y la migración `M0002TenantIsolation`.
- **Cómo funciona:** `TenantContext.ExecuteAsync(companyId, acción)` abre conexión y transacción, ejecuta `set_config('app.company_id', ..., true)` y entrega esa conexión a la acción. En PostgreSQL, `company`, `membership` y `audit_entry` tienen `ENABLE` y `FORCE ROW LEVEL SECURITY` con una política que compara `company_id` con `current_setting('app.company_id', true)`. Sin contexto la variable queda vacía, `nullif(..., '')` produce NULL y ninguna fila coincide: no hay filas visibles, y se registra el intento. API y worker usan el rol restringido; el migrador usa el propietario.
- **Nombres y variables:** `app.company_id` es la variable de sesión; `USING` filtra lo que se puede leer o modificar y `WITH CHECK` filtra lo que se puede insertar; `FORCE` hace que ni el dueño de la tabla se salte la política.
- **Por qué así:** se comparó con un esquema por empresa y con una base por empresa (ADR-003): RLS mantiene un solo esquema y migraciones simples, a cambio de exigir disciplina al fijar el contexto, que `TenantContext` centraliza.
- **Qué aprendes:** `SET LOCAL` (o `set_config(..., true)`) termina con la transacción y evita arrastrar contexto al reutilizar conexiones del pool. RLS y la verificación de membresía resuelven partes distintas; hacen falta ambas.

Credenciales y roles por empresa (2.3; 2.4 pendiente)

- **Qué es:** `Authentication.cs`, `CompanyAuthorization.cs`, `CompanyEndpoints.cs`.
- **Cómo funciona:** ver el ciclo de la sección 2.2. `PasswordHasher` de Identity genera hashes PBKDF2 con sal aleatoria. Si el correo no existe, el login igual hace una verificación lenta con un hash sintético para que el tiempo de respuesta no revele qué correos existen. La política de respaldo exige autenticación; solo salud, OpenAPI y login son anónimos. Las políticas son `company-member` (los tres roles), `accountant` (administrador y contador) y `administrator`; consultan PostgreSQL en cada solicitud.
- **Nombres y variables:** `sub` es el claim con el id del usuario; `MapInboundClaims = false` evita que .NET renombre los claims; `ClockSkew` de 30 s tolera pequeñas diferencias de reloj; `ValidAlgorithms` fija HMAC-SHA256 para impedir que un token declare un algoritmo débil.
- **Por qué así:** no guardar el rol en el token evita permisos desactualizados después de un cambio. Costo: una consulta por solicitud; se aceptó por simplicidad.
- **Qué aprendes:** `AuthorizationHandler<TRequirement>` y políticas con nombre; `WebApplicationFactory` permite probar una política sin publicar endpoints ficticios del dominio.

Alta, configuración y archivo (2.7, 2.8; 2.9 se completó en PR-3)

- **Qué es:** `CompanyManagement`, `PlatformAuthorization`, `OperatorBootstrap` y la migración `M0003`.
- **Cómo funciona:** el migrador crea opcionalmente un operador desde `OPERATOR_EMAIL` / `OPERATOR_PASSWORD`. Solo ese operador puede dar de alta una empresa y su primer administrador (`POST /companies`). El RFC se valida con una expresión regular y un trigger (`company_rfc_immutable`) impide cambiarlo. Una cuenta nueva requiere contraseña de 12 caracteres; una existente conserva su contraseña y recibe la misma forma de respuesta. Los cambios de configuración (`DateToleranceDays`, `AmountTolerance`, `ComplementNextBizDay`) guardan valores anteriores y nuevos más el actor en `audit_entry`, en la misma transacción. Archivar (`POST /companies/{id}/archive`) conserva las filas.
- **Por qué así:** denegar INSERT en auditoría demuestra que la configuración se revierte; la acción y su auditoría deben compartir transacción. El primer arranque con Compose detectó que el migrador necesitaba la imagen de runtime de ASP.NET para usar `PasswordHasher`; se corrigió.
- **Atajo de demo:** enviar la contraseña en el alta simplifica esta demo; producción usaría enlaces de invitación o restablecimiento y gestión de secretos. `.env.example` solo contiene marcadores.
- **Qué aprendes:** `JsonUnmappedMemberHandling.Disallow` rechaza campos desconocidos; los *triggers* de PostgreSQL hacen cumplir reglas que ninguna ruta de código puede saltarse.

Pruebas de aislamiento y ADR (2.5, 2.6)

- **Qué es:** `TenantIsolationChecks`, `IsolationTests` y `docs/adr/ADR-003-tenant-isolation-rls.md`.
- **Cómo funciona:** `information_schema` descubre las tablas con `company_id`; se exige RLS habilitado y forzado y al menos una política. `EndpointDataSource` descubre las rutas; cada ruta protegida debe tener caso exitoso, rechazo anónimo y rechazo entre empresas (o de no operador). Se comparan las filas de B antes y después. Quitar a propósito `membership_isolation` produjo cuatro fallos, incluido el guardián de políticas; restaurarla devolvió la suite a verde.
- **Por qué así:** una tabla o ruta nueva sin prueba de aislamiento hace fallar la compilación, en vez de depender de que alguien se acuerde.
- **Qué aprendes:** quitar una política causa denegación por defecto; un 403 por sí solo no demuestra que exista la política, por eso también se verifica la estructura de PostgreSQL.

Pendiente de este PR: la tarea 2.4 sigue sin marcar. Dos pruebas omitidas conservan los nombres exactos: "Viewer cannot decide" y "Different roles" esperan vínculos reales del grupo 8; "Tolerance changed" espera una corrida del grupo 6. Persistencia y auditoría de configuración sí se prueban ahora; no se afirma el comportamiento de la corrida futura. Ninguna tarea se marca completa por una prueba omitida.

PR-1: Cimientos (tareas 1.1 a 1.5), commit `a10c3ab`

Qué es: un proyecto que compila, arranca y se verifica automáticamente, antes de que exista cualquier lógica de negocio.

Verificado: compilación limpia, 4 de 4 pruebas, revisión de formato limpia, los cuatro servicios sanos en Docker, migración subir/bajar/subir en una base de datos real y la ejecución `41122200072` de GitHub Actions terminó con éxito.

Solución y configuración de compilación compartida (tarea 1.1)

- **Qué es:** `Reconciliation.slnx`, `Directory.Build.props`, `.editorconfig`, `global.json`.

- **Cómo funciona:** `Directory.Build.props` se aplica automáticamente a todos los `.csproj` que están debajo, así un solo lugar define cómo se compila cada proyecto. `TreatWarningsAsErrors` junto con `AnalysisLevel` hacen que los analizadores del compilador bloqueen el código descuidado. `global.json` fija el SDK y el ejecutor de pruebas.
- **Nombres y variables:** `Nullable` activa la verificación de nulos; `TreatWarningsAsErrors` convierte advertencias en errores; `AnalysisLevel` elige qué reglas de calidad se aplican.
- **Qué aprendes:** una *solución* (`.slnx`) agrupa *proyectos*; un *proyecto* (`.csproj`) compila un ensamblado (un `.dll` o un programa). Dos analizadores ya enseñaron algo: renombrar el namespace `Shared` a `Common` (palabra reservada en VB) y usar `[LoggerMessage]` en lugar de logging con cadenas (más rápido y verificado en compilación).

Biblioteca Core: módulos y código común (tarea 1.1)

- **Qué es:** `src/Reconciliation.Core`, con siete carpetas que coinciden con el diseño: Ingestion, Matching, Review, Audit, Tenancy, Sat, Jobs.
- **Cómo funciona:** la API y el worker la referencian, así la lógica se escribe una sola vez. Cada módulo tiene una clase marcadora pequeña que implementa `IModule` (`Common/IModule.cs`). La regla del diseño es que los módulos se comunican por interfaces y nunca por las tablas de otro.
- **Qué aprendes:** una `interface` es un contrato sin código; una *biblioteca de clases* es código sin `Main`, reutilizado por los programas.

Verificación de salud y acceso a la base de datos (tarea 1.2)

- **Qué es:** `IDatabaseHealth`, `NpgsqlDatabaseHealth`, `HealthEndpoint`, `AddReconciliationDatabase`.
- **Cómo funciona:** `NpgsqlDatabaseHealth` ejecuta `SELECT 1`. `MapHealth()` publica `GET /health` y responde 200 con `{"status":"ok","database":"up"}` o 503. Docker, CI y más adelante un balanceador de carga la usan para saber si el programa está vivo y llega a la base. `AddReconciliationDatabase` lee `ConnectionStrings:Default` y falla con un mensaje claro si falta.
- **Nombres y variables:** `NpgsqlDataSource` es el grupo de conexiones compartido; `ConnectionStringName = "Default"`.
- **Qué aprendes:**
  - *Inyección de dependencias:* `AddReconciliationDatabase` registra servicios una vez; ASP.NET se los entrega a quien los pida en un constructor. Las pruebas cambian la verificación real por una falsa.
  - Los *métodos de extensión* (`this IServiceCollection services`) agregan métodos a un tipo que no te pertenece.
  - Los *constructores primarios* (`class NpgsqlDatabaseHealth(NpgsqlDataSource dataSource)`) y los *records* (`record HealthResponse(...)`) eliminan código repetitivo.
  - `await using`, `async`/`await` y `CancellationToken` para llamadas a la base de datos sin bloquear.
  - Una ruta de *minimal API* es simplemente `app.MapGet("/health", ...)`.

Host de la API (tarea 1.2)

- **Qué es:** `src/Reconciliation.Api/Program.cs`, el punto de entrada de los usuarios.

- **Cómo funciona:** pocas líneas con *top-level statements*: construir, registrar servicios, activar autenticación y autorización, mapear endpoints, ejecutar. `public partial class Program;` permite que las pruebas arranquen toda la app en memoria.
- **Qué aprendes:** `WebApplication.CreateBuilder` / `Build` / `Run` es el ciclo de vida estándar de ASP.NET Core.

Host del worker (tarea 1.2)

- **Qué es:** `JobLoop`, un `BackgroundService`, para trabajo largo que no debe bloquear las solicitudes web. También puede arrancarse varias veces para procesar trabajos en paralelo.
- **Cómo funciona:** el host llama a `ExecuteAsync` al arrancar y cancela el token al detenerse. Hoy el ciclo solo escribe en el log y espera; el grupo 4 lo reemplaza por la cola de trabajos real.
- **Qué aprendes:** servicios hospedados; métodos `partial` con el *generador de código* `[LoggerMessage]` (el compilador escribe el código de logging por ti).

Migraciones de la base de datos (tarea 1.5)

- **Qué es:** `Reconciliation.Migrator` y `M0001CompanyUserMembership`.
- **Cómo funciona:** cada migración tiene `Up()` (crear) y `Down()` (deshacer). La herramienta ejecuta `up` o `down` mediante FluentMigrator, que registra las versiones aplicadas en una tabla `VersionInfo`. Devuelve código de salida 0 si todo salió bien y distinto de 0 si falló, para que scripts y CI reaccionen.
- **Notas del esquema:** el RFC se valida con un `CHECK`; las configuraciones tipo dinero usan `numeric(18,2)`, nunca punto flotante; `membership` une a un usuario con una empresa con un rol.
- **Por qué así:** los cambios de la base de datos son código versionado, así todos los entornos reciben el mismo esquema y cualquier cambio se puede deshacer.
- **Qué aprendes:** los *literales de cadena crudos* (`"""..."""`) contienen SQL de varias líneas; una migración nunca debe editarse una vez compartida; se agrega otra.

Stub del SAT (tarea 1.2, marcador)

- **Qué es:** un sustituto del servicio real del SAT para poder construir y probar la consulta de estatus sin conexión.
- **Cómo funciona:** una app ASP.NET vacía con `/health` y un 501 para todo lo demás. La tarea 7.2 la convierte en una simulación real.

Imagen de Docker y Compose (tarea 1.2)

- **Qué es:** un `Dockerfile` con varios *targets* (`api`, `worker`, `migrator`, `sat-stub`) y `docker-compose.yml`.
- **Cómo funciona:** Compose arranca PostgreSQL, espera a que esté sano, ejecuta el migrador una vez y luego arranca el resto. Los contenedores corren con un usuario sin privilegios de root. Un solo comando (`docker compose up --build --wait`) da a cualquiera el sistema completo, idéntico en cada máquina.
- **Nombres y variables:** puertos publicados solo en `127.0.0.1` (API 8080, stub 8081, base 5432); el worker expone su salud en 3001 solo dentro de la red de Compose.
- **Qué aprendes:** las *compilaciones en varias etapas* (imagen grande de SDK para compilar, imagen pequeña de runtime para ejecutar); `depends_on` con `service_healthy` y `service_completed_successfully` describen el orden de arranque.

Pruebas (tareas 1.1 / 1.2)

- **Qué es:** `tests/Reconciliation.Tests`, con xUnit.
- **Cómo funciona:** métodos `[Fact]` y `[Theory]`; `WebApplicationFactory<Program>` ejecuta la API real en memoria con una verificación de base de datos falsa (`HealthEndpointTests.cs`); Testcontainers levanta una PostgreSQL real para las pruebas de multiempresa e ingesta.
- **Qué aprendes:** un *fixture* (`IClassFixture`) comparte entre pruebas una preparación costosa; los falsos mantienen las pruebas rápidas y las bases reales detectan errores de SQL.

CI y ADR (tareas 1.3, 1.4)

- **Qué es:** `.github/workflows/ci.yml` y `docs/adr/ADR-001`, `ADR-002`.
- **Cómo funciona:** en cada push, el CI ejecuta `dotnet format --verify-no-changes`, `dotnet build`, `dotnet test`, migraciones subir/bajar/subir y `docker compose up --build --wait`. Los ADR explican *por qué* se eligieron la estructura y el stack (monolito modular con worker; C# y PostgreSQL) y qué alternativas se descartaron.
- **Qué aprendes:** un pipeline son simplemente los comandos que ejecutarías a mano, escritos. Leer el ADR-002 muestra los compromisos entre TypeScript, Java, Go, PHP, Ruby y Elixir que llevaron a C#.

Hallazgos para recordar de este PR

- Una dependencia de FluentMigrator tenía una vulnerabilidad conocida. Como las advertencias son errores, la compilación se detuvo, y la solución fue fijar una versión corregida (`System.Security.Cryptography.Pkcs` 10.0.14) en lugar de silenciar la advertencia.
- El primer andamiaje fue en TypeScript. Se descartó cuando el stack cambió a C#; nada dependía de él.

6. Siguiente trabajo: grupo 4, Trabajos

Mover los servicios síncronos de ingesta a la cola PostgreSQL conservando transacciones por empresa, resultados por archivo e idempotencia. 2.4 espera decisiones reales del grupo 8; la prueba configuración/corrida espera el grupo 6. El mapa conserva el grupo 2 en progreso de forma intencional.

7. El resto del camino

Grupo	Agrega	Tipos principales
3. Ingesta	Lectura segura de XML de CFDI y CSV bancario, con importación idempotente y verificaciones de documentos y saldos	Módulo, Base de datos
4. Trabajos	Cola de trabajos en PostgreSQL: los workers toman trabajos sin chocar, reintentos, recuperación tras caídas	Módulo, Host, Base de datos
5. Auditoría	Bitácora de solo agregar y encadenada por hash de cada acción, con una función de verificación	Módulo, Base de datos
6. Conciliación	El pipeline de reglas que vincula movimientos con facturas, con motivos, notas de crédito, cancelaciones y fechas límite	Módulo
7. Estatus SAT	El cliente del SAT con stub, caché, reintentos y reproducción, y luego el servicio real	Módulo, Host
8. Revisión + API	Cola de revisión, decisiones, resúmenes, documento OpenAPI y una interfaz mínima	Módulo, API
9. Datos de prueba	Anonimizador, generador de estados de cuenta, casos límite, semilla de demo y escaneo de fugas	Herramienta

Grupo	Agrega	Tipos principales
10. Observabilidad	Logs estructurados, trazas y métricas, pruebas de inyección de fallos, guía de operación	Pipeline, Docs
11. Entrega	ADR del diseño en la nube, vistas de arquitectura, recorrido del README, recorrido grabado	Docs

8. Cómo mantener esta bitácora al día

8. Termina un pull request y marca sus tareas en `tasks.md` (`- [x]`).
9. Ejecuta `python3 tools/progress_diagram.py` y `python3 tools/progress_diagram.py --lang es`, y haz commit de los nuevos `docs/diagrams/progress/status-map*`.
10. Agrega una entrada "PR-n" en la sección 5 con la misma forma: Qué es, Cómo funciona, Nombres y variables, Por qué así, Qué aprendes y Verificado. Si el PR introduce variables de entorno, cabeceras, columnas o tipos nuevos, agrégalos a la sección 3.
11. Actualiza la sección 2 (arquitectura) si cambió un flujo, un módulo o una tabla.
12. Mueve "Siguiente trabajo" (sección 6) al grupo siguiente.
13. Regenera los exportados en `exports/` (Word y PDF) a partir de este archivo; nunca los edites a mano.
14. La plantilla de pull request (`.github/pull_request_template.md`) pide esta misma información, así que la descripción se puede pegar en la bitácora.

9. Glosario breve

- **Host:** un programa que sigue ejecutándose (servidor web, worker en segundo plano).
- **Inyección de dependencias (DI):** el framework crea los objetos que tu clase necesita y se los pasa; son fáciles de reemplazar por falsos en las pruebas.
- **Migración:** un cambio versionado de la base de datos con `Up` y `Down`.
- **Seguridad a nivel de fila (RLS):** una función de PostgreSQL que filtra filas por política, aplicada por la propia base de datos.
- **JWT (JSON Web Token):** una credencial firmada que el cliente presenta en cada solicitud; el servidor comprueba la firma sin consultar una sesión guardada.
- **Claim:** un dato dentro del token (aquí, `sub` con el id del usuario).
- **PBKDF2:** algoritmo lento de hash de contraseñas con sal; lento a propósito para frenar ataques de fuerza bruta.
- **XXE:** ataque en que un XML hace que el servidor lea archivos o URL externas; se evita prohibiendo DTD y resolutores.
- **Multipart:** formato HTTP para enviar archivos y campos juntos en una solicitud.
- **Idempotente:** hacer lo mismo dos veces da el mismo resultado que hacerlo una vez (cargar el mismo archivo dos veces crea un solo registro).
- **Transacción:** un conjunto de cambios que se confirma completo o no se aplica; `COMMIT` / `ROLLBACK`.
- **Testcontainers:** biblioteca que levanta una base de datos real en Docker durante las pruebas.
- **ADR:** registro de decisión de arquitectura; una nota breve sobre una decisión y las alternativas descartadas.

- **Sano (healthy):** la verificación de salud del contenedor pasó, así que los servicios que dependen de él pueden arrancar.